Introduction to Machine Learning Problems: LASSO and Model Selection

Shan Qureshi

September 2026

1. (a)

```python
best_rss = float('inf')
best_j = -1

for j in range(Xtr.shape[1]):
    model = LinearRegression()
    model.fit(Xtr[:, j:j+1], ytr)
    yhat = model.predict(Xts[:, j:j+1])

    rss = np.sum((yts - yhat)**2)
    if rss < best_rss:
        best_rss = rss
        best_j = j
```

(b)

```python
best_rss = float('inf')
best_features = None
p = Xtr.shape[1]

for j1 in range(p):
    for j2 in range(j1 + 1, p):
        X_train_sub = Xtr[:, [j1, j2]]
        X_test_sub = Xts[:, [j1, j2]]

        model = LinearRegression()
        model.fit(X_train_sub, ytr)
        yhat = model.predict(X_test_sub)

        rss = sum((yts - yhat)**2)
        if rss < best_rss:
            best_rss = rss
            best_features = (j1, j2)
```

(c) $\binom{1000}{10} = \frac{1000!}{10! \cdot 990!} \approx 2.63 \times 10^{23}$

2. (a) $\phi(w) = \sum_{j=1}^{N} w_j^2$

   (b) $\phi(w) = \sum_{j=1}^{N} \max(0, -w_j)$

   (c) $\phi(w) = \sum_{j=1}^{N} (w_j - w_{j-1})^2$

   (d) $\phi(w) = \sum_{j=1}^{N} |w_j - w_{j-1}|$

3. (a) The std deviation of income is larger than the std deviation of age. Because of the scale, the corresponding β_1 will be much smaller than β_2.

   (b) $\hat{y} = \left[\bar{y} - s_y \frac{\alpha_1 \bar{x}_1}{s_1} - s_y \frac{\alpha_2 \bar{x}_2}{s_2}\right] + \left[\frac{s_y \alpha_1}{s_1}\right] x_1 + \left[\frac{s_y \alpha_2}{s_2}\right] x_2$
   Plugging in the constants,
   - $\beta_1 = \frac{100 \times 0.6}{15000} = 0.004$
   - $\beta_2 = \frac{100 \times (-0.3)}{10} = -3$
   - $\beta_0 = 300 - (0.004 \times 50000) - (-3 \times 45) = 300 - 200 + 135 = 235$

4.

```
import numpy as np
# Normalize the training data to remove the mean and
    standard deviation from both Xtr and ytr.
X_mean = np.mean(Xtr, axis=0)
X_std = np.std(Xtr, axis=0)
y_mean = np.mean(ytr)
y_std = np.std(ytr)

Ztr = (Xtr - X_mean) / X_std
utr = (ytr - y_mean) / y_std

# Fit the model on the normalized data.
model = SomeModel()
model.fit(Ztr, utr)

# Predict the values yhat on the test data
Zts = (Xts - X_mean) / X_std
uhat = model.predict(Zts)
yhat = (uhat * y_std) + y_mean

# Measure the RSS on the test data.
rss = sum((yts - yhat)**2)
```

5.

```
import numpy as np

# Create p = 100 values of alpha_j uniformly in some
    interval alpha_j in [a, b] where a and b are given.
p = 100
```

```
alphas = np.linspace(a, b, p)

Ztr = np.exp(-Xtr.reshape(-1, 1) * alphas)
Zts = np.exp(-Xts.reshape(-1, 1) * alphas)

# Fit the linear model (3) on the training data for some
    given lam.
model = Lasso(lam=0.1)
beta = model.fit(Ztr, ytr)

# Measure the test error.
yhat = model.predict(Zts)
test_error = np.mean((yts - yhat)**2)

# Find coefficients alpha_j and beta_j corresponding to
    the largest k = 3 values in beta_j . You can use the
    function np.argsort.
k = 3
top_k_indices = np.argsort(np.abs(beta))[-k:]

best_alphas = alphas[top_k_indices]
best_betas = beta[top_k_indices]
```

6. (a) $J'(w) = w - y + \lambda = 0 \rightarrow w = y - \lambda$ For $w > 0$ to be true, $y > \lambda$

   (b) $w = y + \lambda$ For $w < 0$ to be true, $y < -\lambda$

   (c) $\hat{w} = \begin{cases} y - \lambda & \text{if } y > \lambda \\ y + \lambda & \text{if } y < -\lambda \\ 0 & \text{if } -\lambda \leq y \leq \lambda \end{cases}$